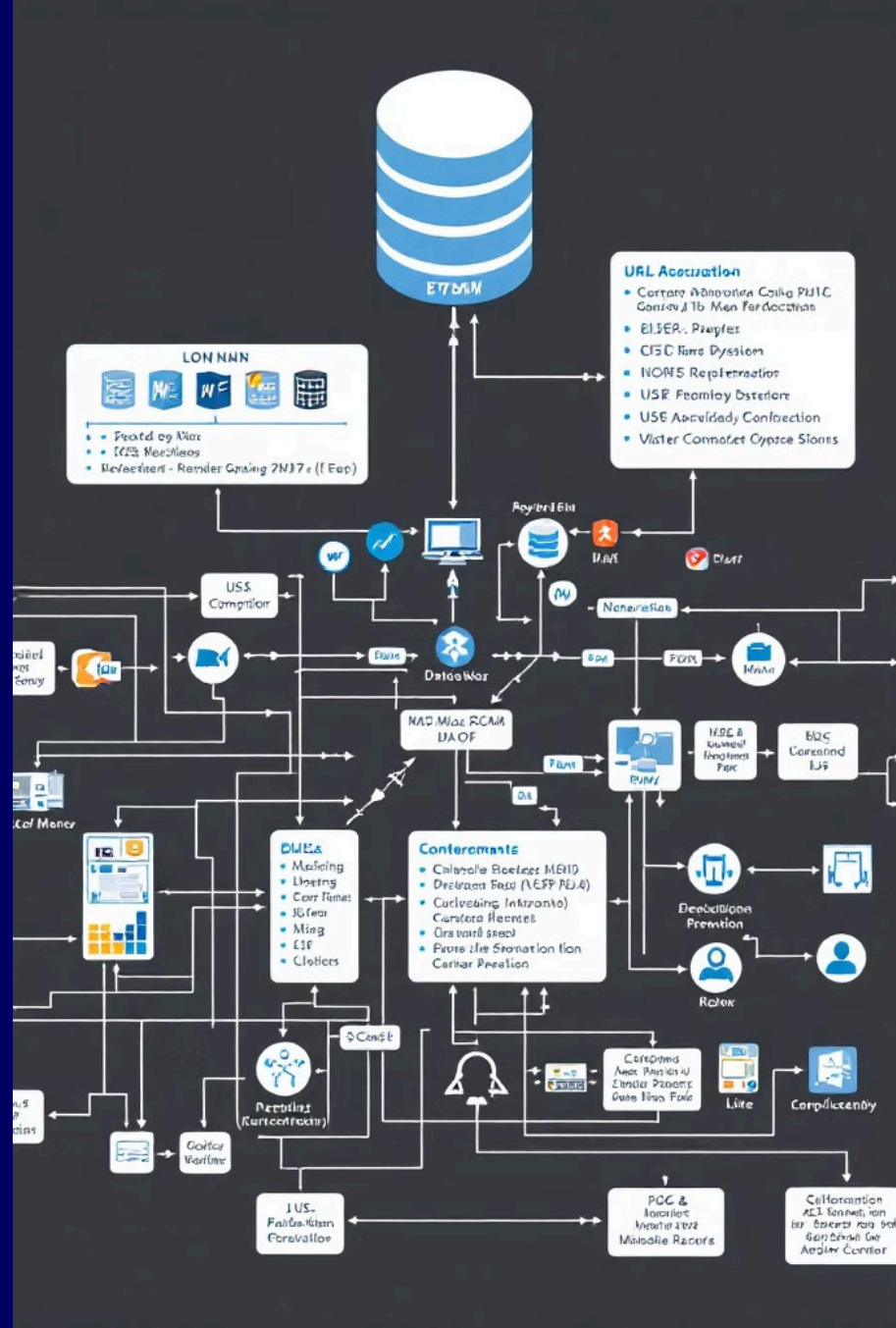
Formação Completa

AWS Developer Associate

Banco de Dados



Bancos de Dados Relacionais vs. Não Relacionais

Bancos Relacionais

Organizam dados em tabelas estruturadas com esquemas rígidos.

- Utilizam linguagem SQL
- Garantem integridade referencial
- Suportam transações ACID
- Exemplo na AWS: Amazon RDS

Bancos Não Relacionais (NoSQL)

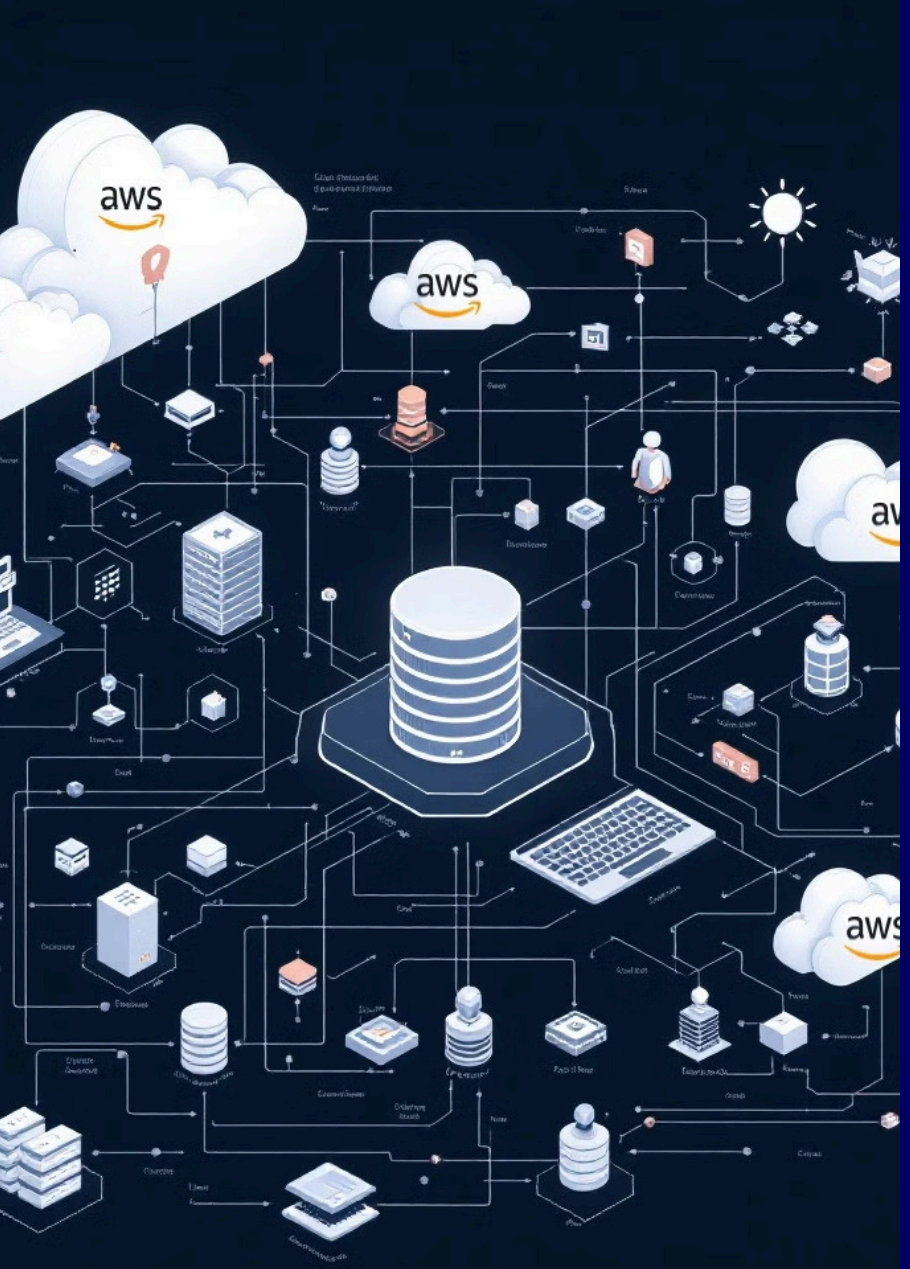
Armazenam dados em formatos flexíveis sem esquema fixo.

- Modelos diversos: documentos, chave-valor, coluna larga
- Alta escalabilidade horizontal
- Flexibilidade de esquema
- Exemplo na AWS: Amazon DynamoDB

Comparação: Características Técnicas



Característica	Bancos Relacionais	Bancos Não Relacionais
Modelo de Dados	Tabelas, linhas e colunas	Tabelas, itens e atributos
Esquema	Rígido, predefinido	Flexível, sem esquema
Consultas	SQL complexo	API simples, sem SQL
Escalabilidade	Vertical (aumentar recursos)	Horizontal (distribuído)
Transações	ACID completo	ACID em nível de item



Bancos de Dados na AWS

Amazon RDS

Serviço gerenciado que facilita a configuração, operação e escalabilidade de bancos de dados relacionais na nuvem.

Características Principais

- Suporte a **transações ACID.**
- **Esquemas rígidos** e **integridade referencial**.
- Execução de **consultas complexas via SQL**.
- **Escalabilidade vertical** (aumento de recursos da instância) e suporte a **replicação para escalabilidade horizontal**.



Mecanismos de Banco de Dados RDS

Aurora (MySQL/PostgreSQL)

Compatível com MySQL e PostgreSQL com performance até 5x maior. Projetado para alta disponibilidade na nuvem AWS.

MySQL, PostgreSQL e MariaDB

Mecanismos de código aberto populares. Oferecem recursos robustos com baixo custo total.

Microsoft SQL Server e Oracle

Soluções corporativas tradicionais Suporte a migrações de aplicações empresariais complexas.

IBM Db2

O Db2 é um banco de dados relacional da IBM que oferece suporte a workloads transacionais e analíticas em grande escala.

Criando uma instância RDS com Python (boto3)

Este exemplo demonstra a criação de um banco MySQL com configurações básicas. Os parâmetros devem ser ajustados conforme suas necessidades.

```
import boto3

rds_client = boto3.client('rds', region_name='us-east-1')

response = rds_client.create_db_instance(
    DBInstanceIdentifier='mydbinstance',
    AllocatedStorage=20,
    DBInstanceClass='db.t2.micro',
    Engine='mysql',
    MasterUsername='admin',
    MasterUserPassword='password123',
    PubliclyAccessible=True
)
print("Resposta da cria  o da inst ncia:", response)
```



Amazon Aurora

Banco de dados relacional compatível com MySQL e PostgreSQL, projetado para oferecer desempenho até 5x maior que o MySQL padrão e 3x maior que o PostgreSQL.

Características Principais

- **Alto desempenho** e **baixa latência**.
- **Armazenamento distribuído** e tolerante a falhas.
- **Escalabilidade automática** até **128 TB**.
- **Replicação rápida** e failover automático.

Amazon DynamoDB: Banco NoSQL

Banco de Dados NoSQL Gerenciado

Serviço de banco de dados não relacional totalmente gerenciado que oferece desempenho rápido e previsível com escalabilidade contínua.

Modelo de Dados Flexível

Armazena dados em formato de documentos (JSON) ou chave-valor, sem necessidade de esquema predefinido, permitindo adaptação rápida às mudanças.

Escalabilidade Horizontal

Escala automaticamente para lidar com volumes massivos de dados e tráfego, distribuindo dados em múltiplos servidores.



Exemplo prático de como criar uma tabela no DynamoDB, inserir e recuperar dados.

```python
import boto3

# Criando o cliente do DynamoDB
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')

# Criando uma tabela chamada 'Usuarios'
table_name = "Usuarios"

try:
    table = dynamodb.create_table(
        TableName=table_name,
        KeySchema=[{'AttributeName': 'id', 'KeyType': 'HASH'}],  # Chave primária
        AttributeDefinitions=[{'AttributeName': 'id', 'AttributeType': 'S'}],  # Tipo String
        ProvisionedThroughput={'ReadCapacityUnits': 5, 'WriteCapacityUnits': 5}
    )
    table.wait_until_exists()
    print(f"Tabela '{table_name}' criada com sucesso!")
except dynamodb.meta.client.exceptions.ResourceInUseException:
    print(f"Tabela '{table_name}' já existe.")

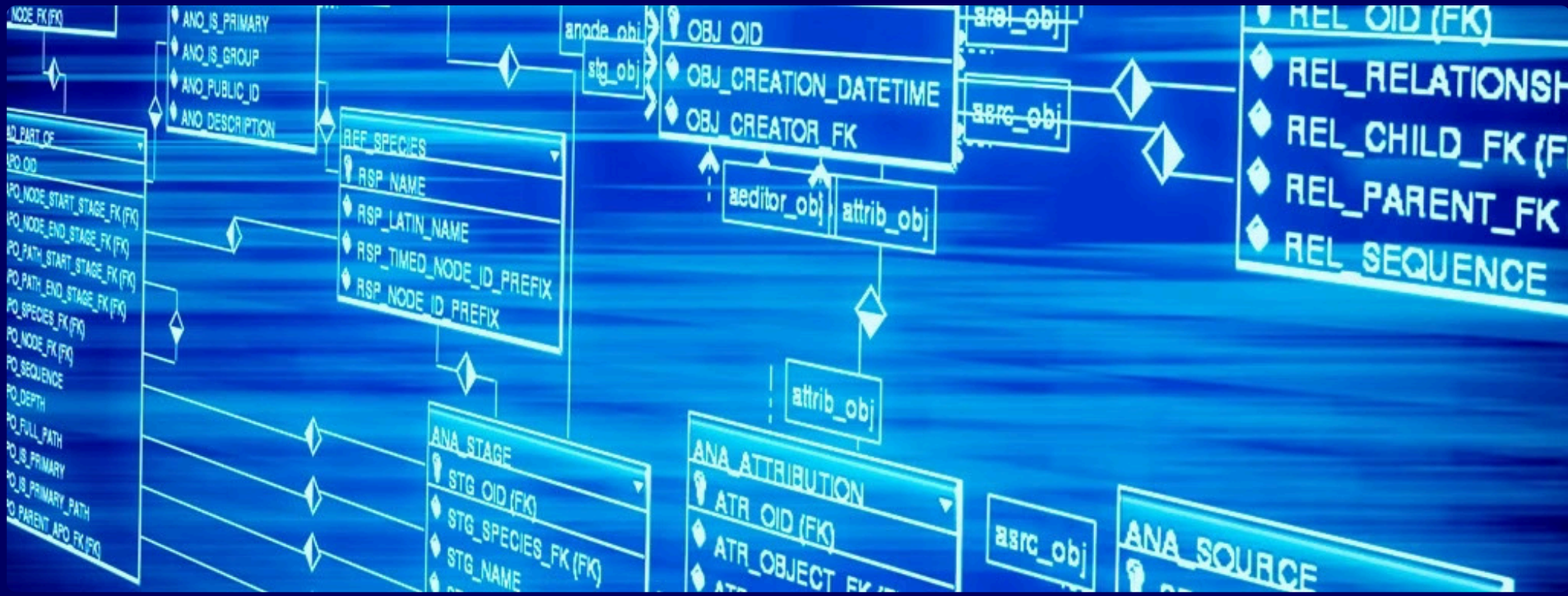
# Obtendo a referência da tabela
table = dynamodb.Table(table_name)

# Inserindo um item na tabela
table.put_item(
    Item={
        'id': '1',
        'nome': 'João Silva',
        'email': 'joao@email.com'
    }
)
print("Item inserido com sucesso!")

# Recuperando o item pelo ID
response = table.get_item(Key={'id': '1'})

if 'Item' in response:
    print("Dados recuperados:", response['Item'])
else:
    print("Item não encontrado.")
```

Comparação: Casos de Uso



Bancos Relacionais (RDS)

Ideal para aplicações com esquema rígido e transações complexas.

Sistemas financeiros, ERPs e aplicações onde a integridade é crítica.

DynamoDB (NoSQL)

Perfeito para aplicações com tráfego intenso e padrões de acesso previsíveis.

Jogos online, IoT, aplicações móveis e cargas de trabalho variáveis.

Fatores de Escolha

Volume de dados, padrões de consulta e requisitos de latência.

Considere também escalabilidade, custos e conhecimento da equipe.

Estrutura de Dados

RDS vs. DynamoDB

Comparando os Modelos

Relacional ou SQL	Document ou NoSQL
Rows (linhas ou registros)	Documentos
Colunas	Propriedades
Schema rígido	Schema-free
Altamente normalizado	Tipicamente denormalizado

Tabela: Funcionário

ID	Nome	Admissão
10	Luis Renato	25/05/2019

Tabela: Férias

Cod	ID	Período	Diretor
201	10	07/2020	Cristiano
202	10	05/2021	Carlos
203	10	09/2022	Carlos

Document

```
{
  "ID": 10,
  "Nome": "Luis Renato",
  "Admissão":"25/05/2019",
  "Férias": [
     {"Período": "07/2020","Diretor":"Cristiano"},
     {"Período": "05/2021","Diretor":"Carlos"},
     {"Período": "09/2022","Diretor":"Carlos"},
   ]
}
```

RDS/Aurora (Relacional)

Dados organizados em tabelas com linhas e colunas. Cada tabela tem um esquema definido que especifica os tipos de dados para cada coluna. Relacionamentos entre tabelas são mantidos através de chaves estrangeiras.

DynamoDB (NoSQL)

Dados armazenados como documentos JSON com atributos flexíveis. Cada item deve ter uma chave primária, mas outros atributos são opcionais. Não há esquema rígido, permitindo que diferentes itens tenham diferentes atributos.

Operações CRUD: Conceitos Básicos

Operações CRUD referem-se ao conjunto básico de ações necessárias para manipular dados em um data store: Criar, Ler, Atualizar e Deletar.

Create

Operação para inserir novos registros no banco de dados.

- RDS: INSERT INTO com SQL estruturado
- DynamoDB: PutItem com JSON flexível

Read

Consulta e recuperação de dados existentes.

- RDS: SELECT com JOINs poderosos
- DynamoDB: GetItem, Query e Scan

Update

Modificação de registros já armazenados.

- RDS: UPDATE com WHERE condicional
- DynamoDB: UpdateItem com expressões

Delete

Remoção de dados do banco.

- RDS: DELETE FROM com integridade referencial
- DynamoDB: DeleteItem sem verificações de relação

CRUD no Amazon DynamoDB com Python (boto3)

```python
import boto3

# Criar recurso do DynamoDB
dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table('MinhaTabela')

# Criar (Create)
table.put_item(Item={'id': '123', 'nome': 'Exemplo'})

# Ler (Read)
response = table.get_item(Key={'id': '123'})
item = response.get('Item')
print("Item lido:", item)

# Atualizar (Update)
table.update_item(
    Key={'id': '123'},
    UpdateExpression='SET #n = :val1',
    ExpressionAttributeNames={'#n': 'nome'},
    ExpressionAttributeValues={':val1': 'Exemplo Atualizado'}
)

# Deletar (Delete)
table.delete_item(Key={'id': '123'})
```

O código realiza operações CRUD (Create, Read, Update, Delete) em uma tabela do **Amazon DynamoDB** usando **boto3**.

CRUD no RDS com Python

```python
import pymysql

# Conexão com RDS
conn = pymysql.connect(
    host='SEU_HOST',
    user='SEU_USUARIO',
    password='SUA_SENHA',
    database='SEU_BANCO'
)

def criar_registro(nome, email, telefone):
    try:
        with conn.cursor() as cursor:
            sql = """INSERT INTO clientes
                     (nome, email, telefone)
                     VALUES (%s, %s, %s)"""
            cursor.execute(sql, (nome, email, telefone))
        conn.commit()
        print("Cliente inserido com sucesso!")
    finally:
        conn.close()
```

Este exemplo demonstra como inserir um novo registro em uma tabela de clientes em um banco de dados MySQL hospedado no Amazon RDS, utilizando a biblioteca pymysql para Python.

Benefícios e Desafios das Operações CRUD

Benefícios

- Atualização dinâmica de dados
- Criação de aplicações interativas
- Gerenciamento simplificado de informações

Desafios

- Tratamento rigoroso de exceções
- Implementação de idempotência
- Prevenção de duplicações em retries

Boas Práticas

- Validação de entrada de dados
- Uso de transações quando apropriado
- Logging de operações críticas

Implementar tratamento de exceções é essencial para evitar duplicações e manter a integridade dos dados. A idempotência garante que operações repetidas não causem efeitos indesejados.

Índices Secundários no DynamoDB

O que são Índices Secundários?

Estruturas de dados adicionais que permitem consultas eficientes em atributos diferentes da chave primária. Facilitam a recuperação de dados por caminhos de acesso alternativos, melhorando a flexibilidade de consultas.

Por que são Necessários?

O DynamoDB otimiza consultas pela chave primária, mas consultas por outros atributos seriam ineficientes sem índices secundários. Eles permitem modelar diferentes padrões de acesso aos dados sem duplicação.

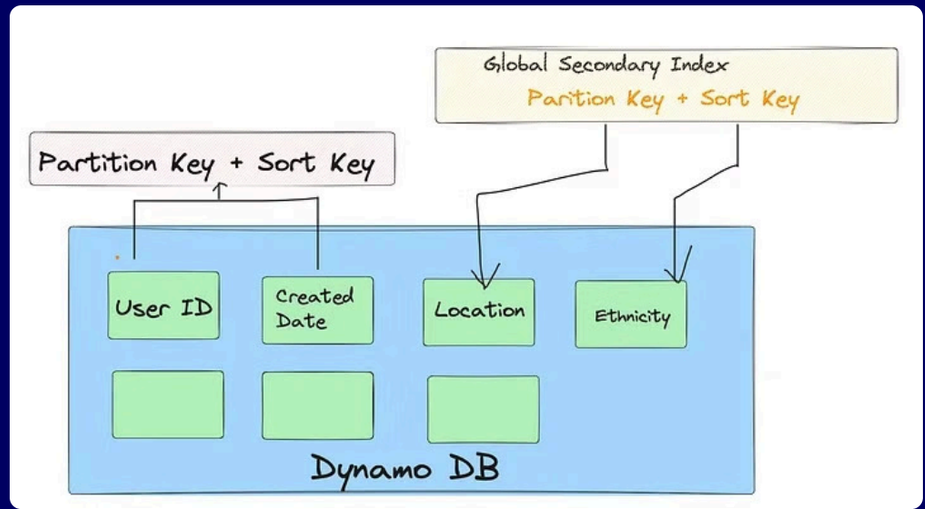
Tipos de Índices Secundários

O DynamoDB oferece dois tipos: Índices Secundários Globais (GSI) e Índices Secundários Locais (LSI). Cada um tem características específicas que determinam seu uso adequado.

Índices Secundários Globais (GSI)

Definição

Índice com uma chave de partição e opcionalmente uma chave de classificação diferentes da tabela base. Abrange todos os itens da tabela e pode ser criado a qualquer momento.



Características

Possui capacidade de leitura/escrita separada da tabela principal. Suporta consistência eventual (não oferece consistência forte). Pode projetar todos ou apenas alguns atributos da tabela base.

Vantagens

Flexibilidade para criar a qualquer momento. Permite consultas eficientes em qualquer atributo. Não tem limitações de tamanho. Suporta até 20 GSIs por tabela.

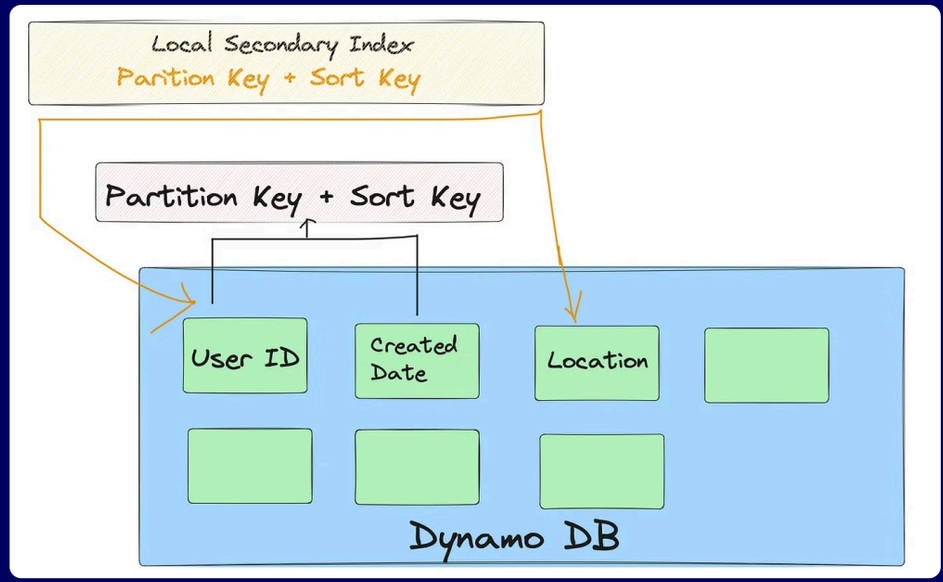
Casos de Uso

Consultas por atributos não-chave. Acesso a dados por múltiplos padrões. Quando a flexibilidade é mais importante que a consistência forte.

Índices Secundários Locais (LSI)

Definição

Índice com a mesma chave de partição da tabela base, mas com uma chave de classificação diferente. Deve ser criado no momento da criação da tabela.



Características

Compartilha a capacidade de leitura/escrita com a tabela principal. Oferece opção de consistência forte. Limitado a 10GB por valor de chave de partição.

Vantagens

Suporta consultas com consistência forte. Permite ordenação alternativa dos itens dentro de uma mesma partição. Útil para relacionamentos um-para-muitos.

Casos de Uso

Quando a consistência forte é essencial. Para consultas que filtram por um atributo de partição específico e ordenam por um atributo diferente.

Comparação: GSI vs. LSI

Característica	Índice Secundário Global (GSI)	Índice Secundário Local (LSI)
Chave de Partição	Diferente da tabela base	Igual à tabela base
Momento de Criação	A qualquer momento	Apenas na criação da tabela
Capacidade	Separada da tabela base	Compartilhada com a tabela base
Consistência	Apenas eventual	Eventual ou forte
Limite por Tabela	20 índices	5 índices
Limite de Tamanho	Sem limite específico	10GB por valor de partição

Índices Secundários no DynamoDB: Resumo

1. **GSI (Global Secondary Index)**

   Permite consultas flexíveis em atributos fora da chave primária, ideal para requisitos dinâmicos.

2. **LSI (Local Secondary Index)**

   Definido na criação da tabela, usa a mesma chave de partição, útil para diferentes ordenações dos dados.

3. **Escolha entre GSI e LSI**

   Depende da necessidade de flexibilidade nas consultas versus restrições na estrutura da tabela.

Modelos de Consistência no DynamoDB

Consistência Eventual

Leituras podem não refletir gravações recentes, mas logo se atualizam.

- Maior throughput e menor custo (½ RCUs)
- Melhor disponibilidade
- Padrão para GSIs
- Ideal para feeds, catálogos e contadores não críticos.

Consistência Forte

Leituras sempre retornam dados mais recentes.

- Garantia de atualização
- Maior consumo de RCUs (2×) e latência
- Não disponível em GSIs
- Essencial para sistemas financeiros e operações críticas

Consistência Eventual vs. Forte: Resumo

Modelos de consistência definem como os dados são visíveis após uma escrita no banco. No **DynamoDB**, há dois tipos principais:

1 **Consistência forte**

Garante que a leitura sempre retorne os dados mais recentes, ideal para aplicações críticas.

2 **Consistência eventual**

Pode retornar dados desatualizados temporariamente, mas melhora desempenho e escalabilidade.

A escolha da consistência ocorre na operação de leitura, como no exemplo ao lado.

```python
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('MinhaTabela')

# Realizando uma leitura com consistência forte
response = table.get_item(
    Key={'id': '123'},
    ConsistentRead=True
)

item = response.get('Item')
print("Item lido com consistência forte:", item)
```

A escolha entre os dois modelos depende dos requisitos de precisão e desempenho da aplicação.

Modo Sob Demanda no DynamoDB

O que é o Modo Sob Demanda?

Opção de faturamento flexível que permite atender a cargas de trabalho imprevisíveis sem planejamento de capacidade. O DynamoDB acomoda automaticamente as workloads à medida que aumentam ou diminuem, cobrando apenas pelas solicitações realmente executadas.

Vantagens

Elimina a necessidade de planejar capacidade. Escala instantaneamente para acomodar picos de tráfego. Simplifica o gerenciamento de custos para aplicações com tráfego variável. Reduz a sobrecarga operacional de monitoramento e ajuste de capacidade.

Considerações de Custo

Geralmente mais caro que o modo provisionado para cargas de trabalho previsíveis. Cobra por solicitação de leitura/escrita em vez de capacidade alocada. Pode ser mais econômico para aplicações com tráfego altamente variável ou imprevisível.



Modo de Capacidade Provisionada no DynamoDB

O modo de capacidade provisionada permite definir antecipadamente a capacidade disponível para sua tabela DynamoDB.

Definição Manual

Você especifica as unidades de capacidade de leitura (RCUs) e escrita (WCUs) necessárias.

Monitoramento Constante

Exige equilíbrio para evitar superprovisionamento (custos elevados) ou subprovisionamento (rejeição de solicitações).

Modelo de Faturamento

Cobrança baseada na capacidade total provisionada, não no consumo real.

Auto Scaling

Você pode usar Auto Scaling para ajustar a capacidade provisionada da tabela automaticamente em resposta às alterações de tráfego.

Quando Usar Cada Opção



DynamoDB com Modo Sob Demanda

Aplicações com tráfego imprevisível ou em desenvolvimento



DynamoDB com Modo Provisionado

Aplicações com tráfego previsível e constante



Amazon Aurora

Aplicações que exigem SQL complexo com alto desempenho



Amazon RDS

Aplicações tradicionais que necessitam de banco relacional

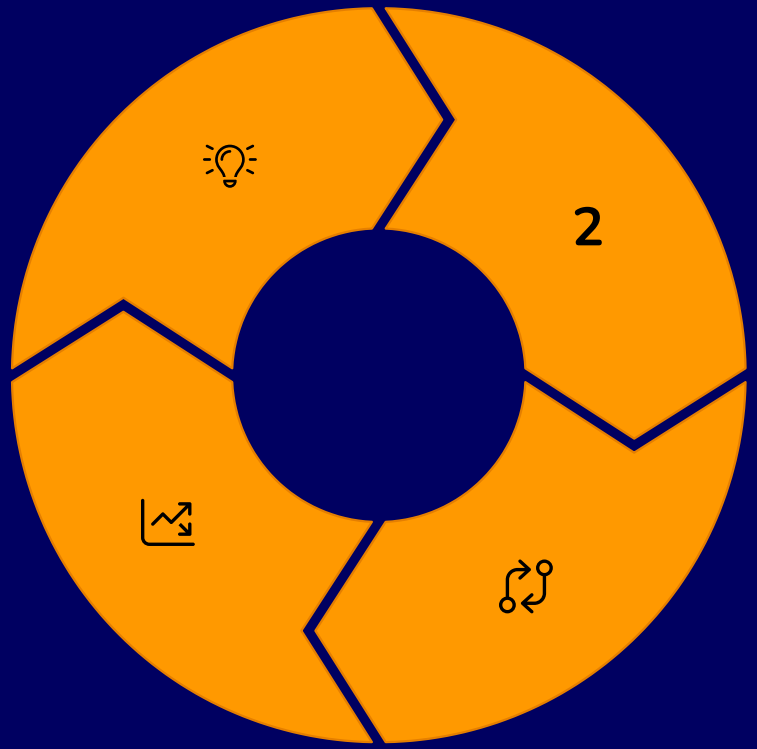
Recomendações

Entenda seu Caso de Uso

Analise padrões de acesso, volume de dados e requisitos de consulta

Monitore e Otimize

Ajuste índices, consistência e capacidade conforme necessário



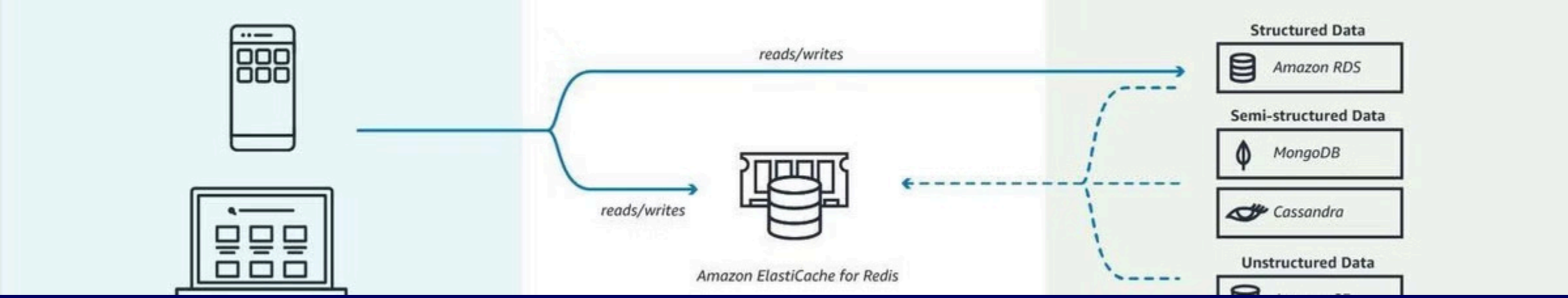
Escolha o Banco Adequado

RDS/Aurora para relacional, DynamoDB para NoSQL

Implemente com Boas Práticas

Utilize os exemplos CRUD e padrões de modelagem adequados

Caching com ElastiCache



O Que é o Armazenamento em Cache?

Na computação, um cache é uma camada de armazenamento de alta velocidade que guarda temporariamente um subconjunto de dados.

Finalidade

Atender solicitações futuras mais rapidamente do que seria possível acessando o local de armazenamento principal.

Eficiência

Permite reutilizar dados recuperados ou computados anteriormente, reduzindo tempo de resposta.

Característica

Geralmente temporário por natureza, armazenando apenas dados frequentemente acessados.

Amazon ElastiCache

O **ElastiCache** acelera aplicações armazenando consultas frequentes, reduzindo a latência. Para evitar dados obsoletos, é essencial configurar **invalidação de cache** e **TTLs adequados**.

Armazenamento em Memória

Guarda dados frequentes em memória RAM para acesso ultrarrápido.

Redução de Carga

Diminui consultas ao banco principal, melhorando a performance geral.

Implementações

Oferece Redis e Memcached como motores de cache gerenciados.



O que é Redis OSS e Memcached no ElastiCache?

Redis

Armazenamento de estruturas de dados avançado com persistência.

- Suporta strings, listas, conjuntos, hashes
- Oferece replicação e alta disponibilidade
- Inclui suporte para transações

Memcached

Sistema de cache simples e eficiente para objetos.

- Modelo mais simples de chave-valor
- Arquitetura multi-thread
- Escalabilidade horizontal facilitada

Quando Escolher Redis

Para aplicações que necessitam de estruturas de dados complexas, persistência ou replicação.

Quando Escolher Memcached

Para casos de uso simples com necessidade de grande escala horizontal distribuída.

Benefícios Comuns

Ambos oferecem acesso sub-milissegundo e redução de carga nos bancos principais.

Valkey: A Evolução do Redis OSS

O que aconteceu?

Em março de 2024, a Redis Inc. anunciou que versões futuras do Redis deixariam de ser open-source.

O que é o Valkey?

- Fork do **Redis OSS 7.2**, criado por desenvolvedores da comunidade.
- Administrado pela **Linux Foundation**, garantindo que permaneça **100% open-source**.
- Recebe **contribuições ativas** de uma comunidade global.

Por que isso é importante?

- **Continuidade**: O Valkey mantém as funcionalidades do Redis OSS.
- **Segurança e estabilidade**: Não depende de uma única empresa.
- **Adoção crescente**: Já integrado a distribuições Linux, provedores de software e **nuvens como a AWS**.

O Valkey já é uma alternativa confiável para substituir o Redis OSS!

Implementando ElastiCache

Exemplo de código para conectar-se ao **Amazon ElastiCache usando Redis** e armazenar temporariamente dados recuperados do banco de dados

```python
import redis

# Conectando ao cluster Redis configurado no ElastiCache
client = redis.Redis(
    host='meu-cluster.redis.cache.amazonaws.com',
    port=6379,
    db=0
)

def get_data(key):
    # Tenta recuperar o valor do cache
    data = client.get(key)

    if data is None:
        # Se não estiver no cache, recupera do banco de dados (simulação)
        data = "dados recuperados"
        client.setex(key, 3600, data)  # Define expiração de 1 hora

    return data

print(get_data("minha_chave"))
```

Resumo: Caching com ElastiCache

1 **Acelera as consultas**

Armazenando dados em memória, reduzindo a latência.

2 **Diminui a carga**

Sobre os bancos de dados primários, melhorando a escalabilidade do sistema.

3 **Requer gerenciamento**

Cuidadoso de invalidação de cache para evitar a entrega de dados obsoletos.

Por que o que vimos sobre Uso de Data Stores em Aplicações é Importante?

(Visão Prática e da Certificação)

Bancos de Dados e Performance

Bancos relacionais garantem consistência forte e transações ACID, ideais para dados estruturados.

NoSQL oferece escalabilidade horizontal e baixa latência, perfeito para grandes volumes de dados.

Operações e Consistência

CRUD forma a base de qualquer sistema de gerenciamento de dados.

A escolha entre consistência eventual ou forte impacta diretamente a latência e disponibilidade.

Otimização Completa

ElastiCache reduz latência e alivia a carga nos bancos de dados primários.

A integração eficiente entre componentes garante sistemas resilientes e de alta performance.

Esse conhecimento é essencial tanto para projetos práticos quanto para certificações AWS, permitindo a escolha da solução ideal para cada aplicação.